

LAPORAN PRATIKUM PEKAN 5 STRUKTUR DATA

“Single Linked List”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “Queue”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 27 April 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 6 Mei 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Pratikum.....	1
BAB II PEMBAHASAN	2
2.1 Single Linked List.....	2
2.2 Single Linked List Impelentation.....	2
2.2.1 NodeSLL.....	2
2.2.2 TambahSLL.....	3
2.2.3 PencarianSLL.....	5
2.2.4 HapusSLL	6
BAB III KESIMPULAN.....	8
3.1 Kesimpulan	8
3.2 Saran.....	8
DAFTAR PUSTAKA	9
Lampiran Tugas	11
1. Deskripsi Tugas.....	11
2. Aturan Penamaan	11
3. Struktur Program.....	11
5. Kriteria Penilaian	13
2. Deadline	14
Java (ADT) Class · Pasien_2511531009.....	15
Java (Main/Driver) Class · RumahSakit_2511531009	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya kita telah meaplikasikan penggunaan struktur data Queue, Stack, Linked List. Walaupun demikian kita menggunakan class-class tersebut karena sudah tertera pada library bawaan Java. Untuk pemahaman lebih lanjut kita dapat menggunakan sistem node yang membutuhkan pemahaman mendalam mengenai bagaimana java mengolah memori untuk dieksekusi.

Sistem tersebut salah satunya ialah Single Linked List, sebuah struktur data linier yang terdiri dari beberapa node yang saling terhubung. Single Linked List merupakan struktur data yang fleksibel, ia tidak harus memindahkan seluruh elemen data.

1.2 Tujuan

1. Memahami konsep Single Linked List.
2. Memahami fungsi node dan pointer dalam Single Linked List.
3. Memahami operasi dasar Single Linked List pada bahasa pemrograman Java.
4. Memahami proses penambahan, pencarian dan penghapusan data pada Single Linked List.

1.3 Manfaat Pratikum

1. Mampu menjelaskan konsep Single Linked List.
2. Mampu menerapkan fungsi node dan pointer dalam Single Linked List.
3. Mampu mengoperasikan syntax dasar Single Linked List pada bahasa pemrograman Java.
4. Mampu menambahkan program penambahan, pencarian dan penghapusan data pada Single Linked List.

BAB II PEMBAHASAN

2.1 Single Linked List

Mengoperasikan sebuah ADT (*Abstract Data Type*) pada java yang berisi node-node yang saling terhubung. Single Linked List terdiri dari dua bagian, yaitu data dan pointer ke simpul/node berikutnya. Struktur ini sangat fleksibel karena tidak memiliki ukuran tetap seperti array, sehingga memungkinkan penambahan atau penghapusan elemen secara dinamis.

Pada implemmentasi SLL pada Java, terdapat beberapa operasi dasar yang harus diperhatikan, seperti penambahan pada posisi awal (*head*), akhir (*tail*), atau posisi tertentu.

2.2 Single Linked List Impelentation

2.2.1 NodeSLL

```
1 package pekan5_2511531009;
2
3 public class NodeSLL_2511531009 {
4
5     // node bagian data
6     int data_1009;
7
8     // pointer to the the next node
9     NodeSLL_2511531009 next_1009;
10
11     // konstruktor menginisialisasi node dengan data
12     public NodeSLL_2511531009(int data_1009) {
13         this.data_1009 = data_1009;
14         this.next_1009 = null;
15     }
16
17 }
```

Ini merupakan konstruktor dari yang memiliki class tersendiri agar mudah untuk diakses di class mana saja. Nilai null pada line 14 memastikan bahwa setiap node tidak akan menunjuk kepada suatu data random.

2.2.2 TambahSLL

```
1 package pekan5_2511531009;
2
3 public class TambahSLL_2511531009 {
4     public static NodeSLL_2511531009 insertAtFront_1009(NodeSLL_2511531009 head_1009, int value_1009) {
5         NodeSLL_2511531009 new_node_1009 = new NodeSLL_2511531009(value_1009);
6         new_node_1009.next_1009 = head_1009;
7         return new_node_1009;
8     }
9
10    // adds node in the end funct
11    public static NodeSLL_2511531009 insertAtEnd_1009(NodeSLL_2511531009 head_1009, int value_1009) {
12        // create node with a value
13        NodeSLL_2511531009 newNode_1009 = new NodeSLL_2511531009(value_1009);
14
15        // if list isEmpty, then node becomes head
16        if (head_1009 == null) {
17            return newNode_1009;
18        }
19
20        // store head to temp. var.
21        NodeSLL_2511531009 last_1009 = head_1009;
22        // find to the last node
23        while (last_1009.next_1009 != null) {
24            last_1009 = last_1009.next_1009;
25        }
26
27        // change the pointer
28        last_1009.next_1009 = newNode_1009;
29        return head_1009;
30    }
31    static NodeSLL_2511531009 getNode_1009(int data_1009) {
32        return new NodeSLL_2511531009(data_1009);
33    }
34
35    static NodeSLL_2511531009 insertPos_1009(NodeSLL_2511531009 headNode_1009, int pos_1009, int val_1009) {
36        NodeSLL_2511531009 head_1009 = headNode_1009;
37        if (pos_1009 < 1) {
38            System.out.println("Invalid position.");
39        }
40        if (pos_1009 == 1) {
41            NodeSLL_2511531009 new_node_1009 = new NodeSLL_2511531009(val_1009);
42            new_node_1009.next_1009 = head_1009;
43            return new_node_1009;
44        } else {
45            while (pos_1009-- != 0) {
```

Method `insertAtFront()` berfungsi untuk membuat node baru menggunakan konstruktor `NodeSLL`. Dimana pointer `next` dari simpul baru tersebut diarahkan kepada linkedlist sebelumnya, yaitu `head_1009`.

Method `insertAtEnd()` pada line 10 mengembalikan nilai `NodeSLL`, yang mengindikasikan penambahan node pada akhir antrian/list. Jika linked list kosong, node baru akan menjadi kepala (*head*), yang dapat diperhatikan pada line 16. Jika tidak maka pointer `next` dari node terakhir akan diarahkan ke node baru.

```
45            while (pos_1009-- != 0) {
46                if (pos_1009 == 1) {
47                    NodeSLL_2511531009 newNode_1009 = getNode_1009(val_1009);
48                    newNode_1009.next_1009 = headNode_1009.next_1009;
49                    headNode_1009.next_1009 = newNode_1009;
50                    break;
51                }
52                headNode_1009 = headNode_1009.next_1009;
53            }
54            if (pos_1009 != 1)
55                System.out.println("Posisi di luar jangkauan");
56        }
57        return head_1009;
58    }
59    public static void printList_1009(NodeSLL_2511531009 head_1009) {
60        NodeSLL_2511531009 curr_1009 = head_1009;
61        while (curr_1009.next_1009 != null) {
62            System.out.print(curr_1009.data_1009 + "-->");
63            curr_1009 = curr_1009.next_1009;
64        }
65        if (curr_1009.next_1009 == null) {
66            System.out.print(curr_1009.data_1009);
67            System.out.println();
68        }
69    }
70 }
```

Method `insertAtPos()` dibuat agar pengguna dapat mencari posisi yang ditentukan dan menyisipkan node baru pada index tersebut. Method ini bekerja dengan cara menambahkan node baru jika index sama dengan satu.

```

71 public static void main(String[] args) {
72     // example linked list 2 -> 3 -> 5 -> 6
73     NodeSLL_2511531009 head_1009 = new NodeSLL_2511531009(2);
74     head_1009.next_1009 = new NodeSLL_2511531009(3);
75     head_1009.next_1009.next_1009 = new NodeSLL_2511531009(5);
76     head_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(5);
77
78     // print original list
79     System.out.println("tambah 1 simpul di depan: ");
80     int data_1009 = 1;
81     head_1009 = insertAtFront_1009(head_1009, data_1009);
82
83     // print updated list
84     printList_1009(head_1009);
85
86     // adds new node to the end
87     System.out.println("tambah 1 simpul di belakang");
88     int data2_1009 = 7;
89     head_1009 = insertAtEnd_1009(head_1009, data2_1009);
90
91     // print updated list
92     printList_1009(head_1009);
93
94     System.out.println("tambah 1 simpul ke data 4: ");
95     int data3_1009 = 4 ;
96     int pos_1009 = 4 ;
97     head_1009 = insertPos_1009(head_1009, pos_1009, data3_1009);
98
99     // print updated list
100    printList_1009(head_1009);
101 }
102 }
103

```

```

tambah 1 simpul di depan: 1-->2-->3-->5-->5
tambah 1 simpul di belakang: 1-->2-->3-->5-->5-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->5-->7

```

2.2.3 PencarianSLL

```
1 package pekan5_2511531009;
2
3 public class PencarianSLL_2511531009 {
4
5     static boolean searchKey_1009(NodeSLL_2511531009 head_1009, int key_1009) {
6         NodeSLL_2511531009 curr_1009 = head_1009;
7         while (curr_1009 != null) {
8             if (curr_1009.data_1009 == key_1009)
9                 return true;
10            curr_1009 = curr_1009.next_1009;
11        }
12        return false;
13    }
14
15    public static void traversal_1009(NodeSLL_2511531009 head_1009) {
16        // start from head
17        NodeSLL_2511531009 curr_1009 = head_1009;
18
19        // search until pointer == null
20        while (curr_1009 != null) {
21            System.out.print(" " + curr_1009.data_1009);
22            curr_1009 = curr_1009.next_1009;
23        }
24        System.out.println("");
25    }
26 }
```

Fungsi searchKey() digunakan untuk memeriksa apakah elemen dengan nilai tertentu ada dalam linked list. Fungsi ini melakukan iterasi melalui seluruh linked list hingga menemukan elemen dengan nilai yang sesuai atau mencapai akhir linked list.

Traversal() digunakan untuk menampilkan seluruh linked list. Fungsi transversal_1009 digunakan untuk mencetak semua elemen dalam linked list dengan cara melakukan iterasi melalui seluruh linked list dan mencetak nilai setiap node.

```
27 public static void main(String[] args) {
28     NodeSLL_2511531009 head_1009 = new NodeSLL_2511531009(14);
29     head_1009.next_1009 = new NodeSLL_2511531009(21);
30     head_1009.next_1009.next_1009 = new NodeSLL_2511531009(13);
31     head_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(30);
32     head_1009.next_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(10);
33     System.out.print("Penelusuran SLL : ");
34     traversal_1009(head_1009);
35
36     // data that will be searched
37     int key_1009 = 30;
38     System.out.print("cari data " + key_1009 + " = ");
39     if (searchKey_1009(head_1009, key_1009))
40         System.out.println("data ditemukan");
41     else
42         System.out.println("data tidak ditemukan");
43 }
44 }
45 }
```

```
Penelusuran SLL : 14 21 13 30 10
cari data 30 = data ditemukan
```

2.2.4 HapusSLL

```
1 package pekan5_2511531009;
2
3 public class HapusSLL_2511531009 {
4
5     // funtion to delete head node
6     public static NodeSLL_2511531009 deleteHead_1009(NodeSLL_2511531009 head_1009) {
7         // if SLL is empty
8         if (head_1009 == null)
9             return null;
10
11         // move to the next node
12         head_1009 = head_1009.next_1009;
13
14         // return new head
15         return head_1009;
16     }
17
18     // function to delete last node
19     public static NodeSLL_2511531009 removeLastNode(NodeSLL_2511531009 head_1009) {
20         // if the list is empty, return null value
21         if (head_1009 == null) {
22             return null;
23         }
24
25         //if list contains one node, delete node and return null
26         if (head_1009.next_1009 == null) {
27             return null;
28         }
29
30         // search second last node
31         NodeSLL_2511531009 secondLast_1009 = head_1009;
32         while (secondLast_1009.next_1009.next_1009 != null) {
33             secondLast_1009 = secondLast_1009.next_1009;
34         }
35
36         // delete last node
37         secondLast_1009.next_1009 = null;
38         return head_1009;
39     }
40 }
```

Seperti halnya dengan LinkedList, kita juga ingin membuat operasi untuk menghapus sebuah atau beberapa elemen pada Single Linked List. Sebagai contohnya deleteHead() berguna untuk menghapus data pada sebuah kepala di Single Linked List. deleteNode_1009 menghapus pada posisi tertentu, dan removeLastNode untuk menghapus elemen terakhir.

```
41 // function to delete node in a certain position
42 public static NodeSLL_2511531009 deleteNode_1009(NodeSLL_2511531009 head_1009, int pos_1009)
43 {
44     NodeSLL_2511531009 temp_1009 = head_1009;
45     NodeSLL_2511531009 prev_1009 = null;
46
47     // if linked list is null
48     if (temp_1009 == null)
49         return head_1009;
50
51     // case 1 : head is deleted
52     if (pos_1009 == 1) {
53         head_1009 = temp_1009.next_1009;
54         return head_1009;
55     }
56
57     // kasus 2 : menghapus node di tengah
58     // search to the targeted node that will to be deleted
59     for (int i_1009 = 1; temp_1009 != null && i_1009 < pos_1009; i_1009++) {
60         prev_1009 = temp_1009;
61         temp_1009 = temp_1009.next_1009;
62     }
63
64     // if founded, delete node
65     if (temp_1009 != null) {
66         prev_1009.next_1009 = temp_1009.next_1009;
67     } else {
68         System.out.println("Data tidak ditemukan");
69     }
70     return head_1009;
71 }
```

```

71 // function to print SLL
72 public static void printList_1009(NodeSLL_2511531009 head_1009) {
73     NodeSLL_2511531009 curr_1009 = head_1009;
74     while (curr_1009.next_1009 != null) {
75         System.out.print(curr_1009.data_1009 + "-->");
76         curr_1009 = curr_1009.next_1009;
77     }
78     if (curr_1009.next_1009 == null) {
79         System.out.print(curr_1009.data_1009);
80     }
81     System.out.println();
82 }
83
84 // Driver/Main class
85 public static void main(String[] args) {
86     // Create SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
87     NodeSLL_2511531009 head_1009 = new NodeSLL_2511531009(1);
88     head_1009.next_1009 = new NodeSLL_2511531009(2);
89     head_1009.next_1009.next_1009 = new NodeSLL_2511531009(3);
90     head_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(4);
91     head_1009.next_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(5);
92     head_1009.next_1009.next_1009.next_1009.next_1009.next_1009 = new NodeSLL_2511531009(6);
93
94     // print early list
95     System.out.println("list awal : ");
96     printList_1009(head_1009);
97
98     // delete head
99     head_1009 = deleteHead_1009(head_1009);
100    System.out.println("List setelah simpul terakhir di hapus : ");
101    printList_1009(head_1009);
102
103    // deleteing node at pos[2]
104    int pos2_1009 = 2;
105    head_1009 = deleteNode_1009(head_1009, pos2_1009);
106    // print list after deletion
107    System.out.println("List setelah posisi 2 dihapus : ");
108    printList_1009(head_1009);
109 }
110 }

```

```

list awal :
1-->2-->3-->4-->5-->6
List setelah simpul terakhir di hapus :
2-->3-->4-->5-->6
List setelah posisi 2 dihapus :
2-->4-->5-->6

```

BAB III

KESIMPULAN

3.1 Kesimpulan

Beberapa hal yang perlu kita ketahui mengenai Single Linked List adalah bahwa Java memungkinkan pengelolaan data secara dinamis tanpa batasan ukuran tetap seperti array. Operasi-operasi dasar seperti penambahan, penghapusan, dan pencarian dapat dilakukan dengan efisien menggunakan pointer. Namun, pencarian elemen dalam SLL memiliki kompleksitas waktu $O(n)$, karena harus dilakukan secara linier.

3.2 Saran

Seperti biasa, mahasiswa sebaiknya dapat diarah dengan lebih baik selama praktikum berlangsung dengan metode pembelajaran yang lebih *engaging* dan interaktif, untuk menghindari suasana yang monoton dan canggung.

DAFTAR PUSTAKA

- [1] W3Schools, "Singly Linked Lists - DSA Try It Editor," *W3Schools*, 2026.
[Daring]. Tersedia pada:
https://www.w3schools.com/dsa/trydsa.php?filename=demo_linkedLists_singly. [Diakses: May 6, 2026].

LAPORAN TUGAS PEKAN 5 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2026

Lampiran Tugas

Topik : Implementasi Single Linked List

Tema : Simulasi Antrian Rumah Sakit

1. Deskripsi Tugas

Mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. Aturan Penamaan

- a) Nama Kelas : concat sufiks “_” + NIM
Contoh : Pasien_2511531009 dan RumahSakit_2511531009
- b) Nama Variabel dan Method : concat sufiks “_” + NIM mod 10^4
Contoh : String namaPasien_1009, String penyakit_1009
- c) Pointer Next : Pasien_2511531009 next_1009

3. Struktur Program

1. Kelas ADT Pasien (Pasien_2511531009)

Atribut :

- Nama Pasien
- Keluhan / Penyakit
- Nomor Antiran (int)
- Pointer next ke node Pasien berikutnya

Method :

- Constructor untuk menginisialisasi semua atribut
- Selektor (Getter) untuk semua atribut
- Mutator (Setter) untuk setiap atribut

2. Kelas Driver (RumahSakit_2511531009)

- Daftarkan Pasien (Insert at Tail) : Menambahkan node pasien baru linked list. Jika list kosong, node baru langsung menjadi head. Gunakan sebuah variabel counter untuk menyimpan nomor antrian terakhir. Setiap kali fungsi Daftarkan Pasien dipanggil, nomor antrian akan bertambah secara otomatis (auto-increment).
- Panggil Pasien (Delete Head) : Menghapus node terdepan dan menampilkan data pasien yang

- Tampilkan Antrian (Display) : dipanggil. Head digeser ke node berikutnya.
- Cari Pasien (Search) : Menelusuri linked list dari head hingga null dan menampilkan semua data pasien beserta posisinya.
- Cek Status Antrian : Pencarian dilakukan berdasarkan nama pasien secara **CaseInsensitive** (tidak membedakan huruf besar dan kecil). **Tampilkan pesan jika tidak ditemukan.**
- : Menampilkan jumlah total pasien dan informasi pasien terdepan. **Tampilkan pesan list kosong.**

4. Contoh Tampilan Output Program

=== Antrian Rumah Sakit NIM: 2411531005 ===

1. Daftarkan Pasien (Insert)
 2. Panggil Pasien (Delete Head)
 3. Tampilkan Antrian (Display)
 4. Cari Pasien (Search)
 5. Cek Status Antrian
 6. Keluar
- Pilihan: 1
 Masukkan Nama Pasien : Budi Santoso
 Masukkan Keluhan : Demam
 Pasien berhasil didaftarkan! Nomor Antrian: 1

5. Kriteria Penilaian

1. Ketepatan struktur dan pemahaman konsep Single Linked List (pointer next, FIFO).
2. Kesesuaian aturan penamaan kelas dan variabel.
3. Kelengkapan dan kebenaran semua operasi yang diminta.
4. Penanganan kondisi list kosong (tidak boleh error).
5. Kerapihan kode dan pemberian komentar pada bagian logika linked list.
6. Kode tugas dicommit ke github.
7. Pastikan laporan tugas ditambahkan pada laporan praktikum dibuat dengan rapi dan penjelasan yang detail serta tampilkan screenshot outputnya.

2. Deadline

Tugas dikumpulkan paling lambat pada hari Minggu, tanggal 6 Mei 2026, pukul 23.59 WIB.

Java (ADT) Class · Pasien_2511531009

Kelas ini menginisialisasikan node dan pointer untuk digunakan pada class driver.

```
1 package pekan5_2511531009;
2
3 public class Pasien_2511531009 {
4
5     // Atribut
6     String namaPasien_1009;
7     String keluhan_1009;
8     int nomorAntrian_1009;
9
10    // pointer ngab
11    Pasien_2511531009 next_1009;
12
13    // Constructor
14    public Pasien_2511531009(String nP_1009, String kel_1009, int noAnt_1009) {
15        this.namaPasien_1009 = nP_1009;
16        this.keluhan_1009 = kel_1009;
17        this.nomorAntrian_1009 = noAnt_1009;
18        this.next_1009 = null;
19    }
20
21    // Gettersssssssssssssss
22    public String getNamaPasien_1009() {
23        return namaPasien_1009;
24    }
25
26    public String getKeluhan_1009() {
27        return keluhan_1009;
28    }
29
30    public int getNomorAntrian_1009() {
31        return nomorAntrian_1009;
32    }
33
34    // Settersssssssssssssss
35    public void setNamaPasien_1009(String namaPasien_1009) {
36        this.namaPasien_1009 = namaPasien_1009;
37    }
38
39    public void setKeluhan_1009(String keluhan_1009) {
40        this.keluhan_1009 = keluhan_1009;
41    }
42 }
```

Kelas ini berfungsi sebagai untuk mengelola opera

Java (Main/Driver) Class · RumahSakit_2511531009

Kelas ini berfungsi sebagai untuk mengelola operasi pada linked list, seperti menambahkan pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan memeriksa status antrian.

```
1 package pekan5_2511531009;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511531009 {
5
6     private Pasien_2511531009 head_1009; // Pointer ke head
7     private int counter_1009 = 0; // Counter untuk nomor antrian
8
9     // Mendaftarkan Pasien (Insert at Tail)
10    public void daftarkanPasien_1009(String namaPasien_1009, String keluhan_1009) {
11        Pasien_2511531009 newNode_1009 = new Pasien_2511531009(namaPasien_1009, keluhan_1009, ++counter_1009);
12
13        if (head_1009 == null) {
14            head_1009 = newNode_1009;
15        } else {
16            Pasien_2511531009 temp_1009 = head_1009;
17            while (temp_1009.next_1009 != null) {
18                temp_1009 = temp_1009.next_1009;
19            }
20            temp_1009.next_1009 = newNode_1009;
21        }
22        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_1009);
23    }
24
25    // Panggil Pasien (Delete Head)
26    public void panggilPasien_1009() {
27        if (head_1009 == null) {
28            System.out.println("Antrian kosong!");
29        } else {
30            Pasien_2511531009 pasienDipanggil_1009 = head_1009;
31            head_1009 = head_1009.next_1009;
32            System.out.println("Pasien dipanggil: " + pasienDipanggil_1009.getNamaPasien_1009() +
33                ", Keluhan: " + pasienDipanggil_1009.getKeluhan_1009() +
34                ", Nomor Antrian: " + pasienDipanggil_1009.getNomorAntrian_1009());
35        }
36    }
37
38    // Tampilkan Antrian (Display)
39    public void tampilkanAntrian_1009() {
40        if (head_1009 == null) {
41            System.out.println("Antrian kosong!");
42        } else {
43            Pasien_2511531009 temp_1009 = head_1009;
44            System.out.println("=== Daftar Antrian ===");
45            while (temp_1009 != null) {
46                System.out.println("Nomor Antrian: " + temp_1009.getNomorAntrian_1009() +
47                    ", Nama: " + temp_1009.getNamaPasien_1009() +
48                    ", Keluhan: " + temp_1009.getKeluhan_1009());
49                temp_1009 = temp_1009.next_1009;
50            }
51        }
52    }
53 }
```

```

54 // Cari Pasien (Search)
55 public void cariPasien_1009(String namaPasien_1009) {
56     if (head_1009 == null) {
57         System.out.println("Antrian kosong!");
58     } else {
59         Pasien_2511531009 temp_1009 = head_1009;
60         boolean found_1009 = false;
61         while (temp_1009 != null) {
62             if (temp_1009.getNamaPasien_1009().equalsIgnoreCase(namaPasien_1009)) {
63                 System.out.println("Pasien ditemukan: " +
64                     "Nomor Antrian: " + temp_1009.getNomorAntrian_1009() +
65                     ", Nama: " + temp_1009.getNamaPasien_1009() +
66                     ", Keluhan: " + temp_1009.getKeluhan_1009());
67                 found_1009 = true;
68                 break;
69             }
70             temp_1009 = temp_1009.next_1009;
71         }
72         if (!found_1009) {
73             System.out.println("Pasien tidak ditemukan!");
74         }
75     }
76 }
77
78 // Cek status antrian
79 public void cekStatusAntrian_1009() {
80     if (head_1009 == null) {
81         System.out.println("Antrian kosong!");
82     } else {
83         System.out.println("Total Pasien: " + counter_1009);
84         System.out.println("Pasien Terdepan: " +
85             "Nomor Antrian: " + head_1009.getNomorAntrian_1009() +
86             ", Nama: " + head_1009.getNamaPasien_1009() +
87             ", Keluhan: " + head_1009.getKeluhan_1009());
88     }
89 }
90

```

```

91 public static void main(String[] args) {
92     Scanner scanner_1009 = new Scanner(System.in);
93     RumahSakit_2511531009 rumahSakit_1009 = new RumahSakit_2511531009();
94     int pilihan_1009;
95
96     do {
97         System.out.println("=== Antrian Rumah Sakit NIM: 2511531009 ===");
98         System.out.println("1. Daftarkan Pasien (Insert)");
99         System.out.println("2. Panggil Pasien (Delete Head)");
100        System.out.println("3. Tampilkan Antrian (Display)");
101        System.out.println("4. Cari Pasien (Search)");
102        System.out.println("5. Cek Status Antrian");
103        System.out.println("6. Keluar");
104        System.out.print("Pilihan: ");
105        pilihan_1009 = scanner_1009.nextInt();
106        scanner_1009.nextLine(); // this is important >:|
107
108        switch (pilihan_1009) {
109            case 1:
110                System.out.print("Masukkan Nama Pasien: ");
111                String namaPasien_1009 = scanner_1009.nextLine();
112                System.out.print("Masukkan Keluhan: ");
113                String keluhan_1009 = scanner_1009.nextLine();
114                rumahSakit_1009.daftarkanPasien_1009(namaPasien_1009, keluhan_1009);
115                break;
116            case 2:
117                rumahSakit_1009.panggilPasien_1009();
118                break;
119            case 3:
120                rumahSakit_1009.tampilkanAntrian_1009();
121                break;
122            case 4:
123                System.out.print("Masukkan Nama Pasien yang Dicari: ");
124                String namaCari_1009 = scanner_1009.nextLine();
125                rumahSakit_1009.cariPasien_1009(namaCari_1009);
126                break;
127            case 5:
128                rumahSakit_1009.cekStatusAntrian_1009();
129                break;
130            case 6:
131                System.out.println("Terima kasih telah menggunakan program ini!");
132                break;
133            default:
134                System.out.println("Pilihan tidak valid!");
135        }
136        System.out.println();
137    } while (pilihan_1009 != 6);
138
139    scanner_1009.close();
140 }
141 }

```

```

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: bob
Masukkan Keluhan: sakit kepala
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: beni
Masukkan Keluhan: sakit hati
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian ===
Nomor Antrian: 1, Nama: bob, Keluhan: sakit kepala
Nomor Antrian: 2, Nama: beni, Keluhan: sakit hati

```

```

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang Dicari: beni
Pasien ditemukan: Nomor Antrian: 2, Nama: beni, Keluhan: sakit hati

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang Dicari: beno
Pasien tidak ditemukan!

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Total Pasien: 2
Pasien Terdepan: Nomor Antrian: 1, Nama: bob, Keluhan: sakit kepala

```

```

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 7
Pilihan tidak valid!

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil: bob, Keluhan: sakit kepala, Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian ===
Nomor Antrian: 2, Nama: beni, Keluhan: sakit hati

```

```

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil: beni, Keluhan: sakit hati, Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Antrian kosong!

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Antrian kosong!

=== Antrian Rumah Sakit NIM: 2511531009 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih telah menggunakan program ini!

```